

PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

HostelSpace

Hostel Management System

DELIVERABLE 2
SOFTWARE REQUIREMENTS SPECIFICATION

Item	Details
Project Title	HostelSpace
Assigned Project	Hostel Management System
Group Number	Team 11
Course	UE24CS341A : Software Engineering
Methodology	Agile
Version	1.0
Date	2 September 2026

Team Members

USN	SRN / PRN	Name
PES1UG24CS076	PES1202402178	Anvi Srinivas
PES1UG24CS079	PES1202402179	Arin Singh
PES1UG24CS086	PES1202400020	Aryan Singh
PES1UG24CS096	PES1202403356	Atiksh Gour

2.1 Introduction and Purpose

This SRS defines the requirements for HostelSpace. It will guide design, coding, testing, and evaluation. It is written for the project team, faculty evaluator, and the hostel users listed below.

2.1.1 Product Scope

HostelSpace is a browser-based hostel management system. The first version covers student profiles, notices and rules, study-room open/closed status, assigned laundry-day tracking, leave, location-verified attendance, maintenance complaints, mess allocation and coupons, hostel fee payment records, and deviation-weightage records.

It does not include study-room booking, student selection of laundry days, continuous location tracking, full hostel accounting, biometric devices, inventory management, or a native mobile application.

2.2 Product Description

2.2.1 Intended Users and User Classes

User Class	Main Use
Student	Uses student services and views only personal records.
Warden	Reviews leave, views attendance, and records deviations.
Chief Warden	Supervises multiple wardens and handles escalated matters.
Hostel Director	Controls authorized users, information, settings, and reports.
Maintenance Staff	Updates assigned maintenance complaints.
Mess Staff	Maintains mess allocation and coupons.
Laundry Staff	Maintains assigned laundry days and handover/collection records.

2.2.2 Product Functions

- Role-based sign-in and student profile access.
- Hostel notices, contacts, information, and rules.
- Study-room status and assigned laundry tracker.
- Leave, attendance, complaints, mess/coupons, fee payment, and deviation records.
- Authorized staff updates and Hostel Director reports.

2.3 Operating Environment, Constraints, and Assumptions

- Modern browser on a phone, tablet, laptop, or desktop.
- Next.js with JavaScript, HTML, CSS, simple role-based authentication, and an SQL database.
- Agile planning with Jira and version control with Git and GitHub.
- One Chief Warden supervises multiple wardens.
- Students cannot book the study room or select a laundry day.
- Location permission is requested only when attendance is submitted.
- Payment status and reference are stored; payment credentials are not stored.

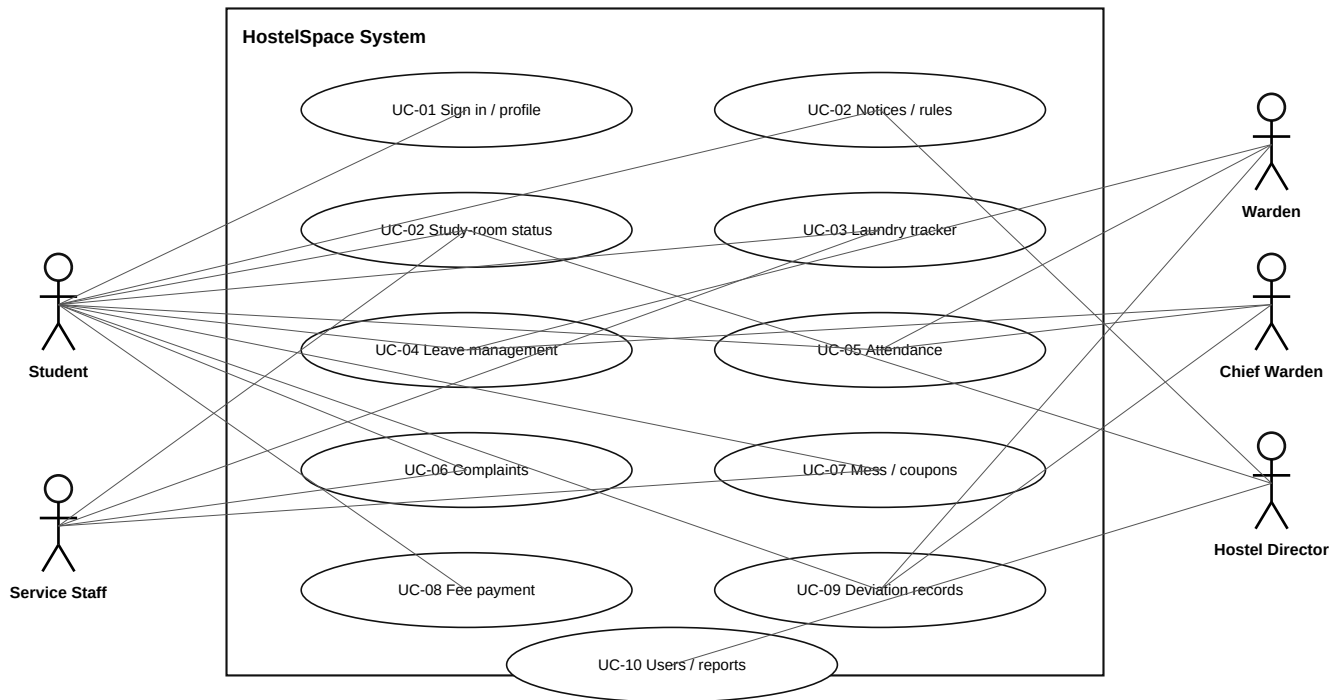
2.3.1 External Interface Requirements

Interface	Requirement
User interface	Sign-in page, role dashboard, clear forms, status displays, and error messages.
Browser ↔ Next.js	Display pages and submit form data.
Next.js ↔ SQL	Store and retrieve users, requests, service records, and reports.
Browser location service	Return location only for attendance submission.
Payment service	Return success, failed, or pending status and a reference.
Communication	Use HTTPS for private communication in deployment.
Hardware	No special hostel hardware is required.

2.4 Use Case / Analysis Models

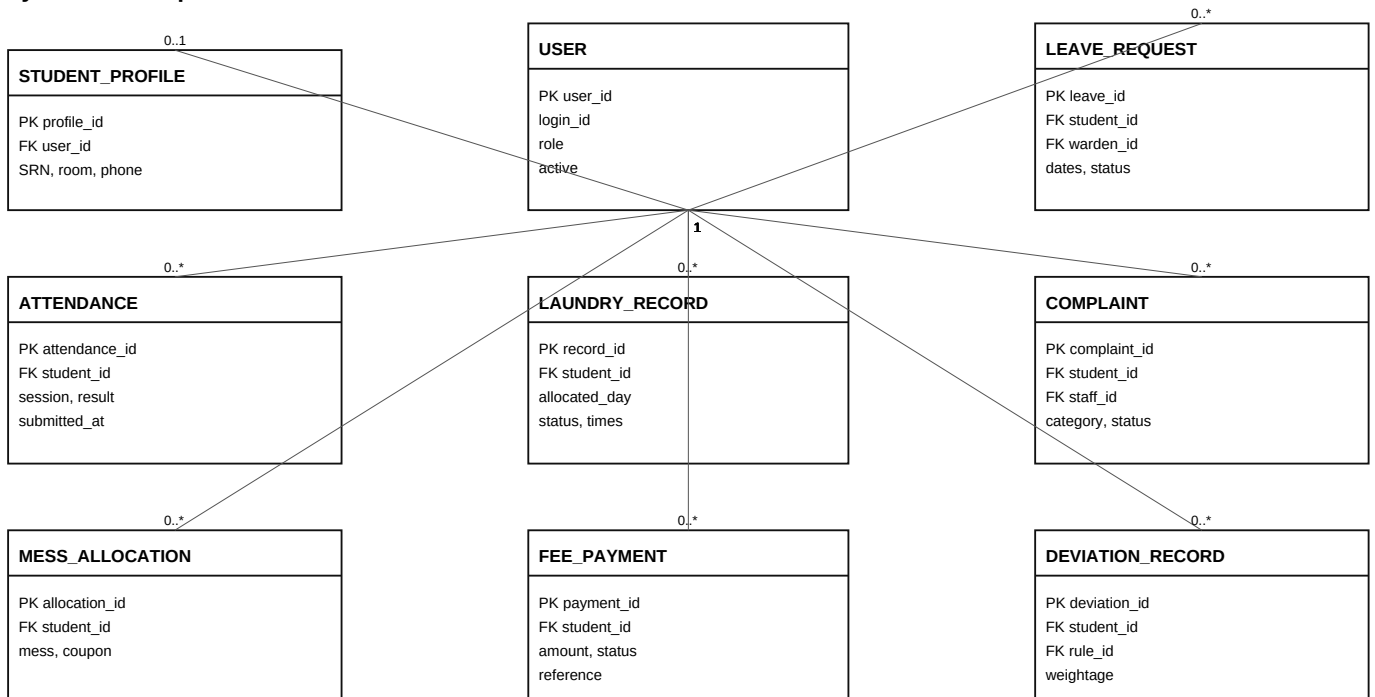
2.4.1 UML Use-Case Diagram

UML Use-Case Diagram



2.4.2 Entity-Relationship Model

Entity-Relationship Model



2.5 Functional Requirements

ID	Combined Functional Requirement	Use Case
FR-01	Provide role-based sign-in, sign-out, dashboard access, and permitted student-profile updates.	UC-01
FR-02	Allow authorized staff to maintain notices, contacts, information, and rules while students view active content.	UC-02
FR-03	Allow authorized staff to set study-room Open/Closed status with optional timings/message; show it to students without booking.	UC-02
FR-04	Allow staff to assign a laundry day and record clothes received/collected; show the assigned day and status to the student without day selection.	UC-03
FR-05	Allow students to submit valid leave details; allow an assigned warden to approve, reject, or escalate; allow the Chief Warden to review escalations.	UC-04
FR-06	Allow staff to configure attendance time/location; check location only during submission; reject invalid attempts and prevent duplicate accepted attendance.	UC-05
FR-07	Allow students to submit complaints, authorized staff to update status, and students to track only their own complaints.	UC-06
FR-08	Allow Mess Staff to maintain allocations/coupons and students to view their current records.	UC-07
FR-09	Allow students to view fees and start payment; store status/reference, keep failed fees unpaid, prevent duplicate success, and store no payment credentials.	UC-08
FR-10	Allow wardens to record rule deviations/weightage; total active entries; show students only their own entries.	UC-09
FR-11	Allow the Hostel Director to manage authorized users, information, settings, and required summary reports.	UC-10

2.6 Non-Functional Requirements

ID	Requirement
NFR-01	Normal pages should load within 3 seconds during the agreed project test load.
NFR-02	Forms shall use clear labels and correction messages.
NFR-03	Retries shall not create duplicate attendance or payment records.
NFR-04	The system shall work on current Chrome, Edge, and Firefox and keep modules easy to test.
NFR-05	Important validation and calculation rules shall have automated tests.

2.6.1 Security Requirements

ID	Requirement
SEC-01	Every protected request shall check the active session, role, and record owner.
SEC-02	Passwords shall use the authentication system's secure hashing method.
SEC-03	Students shall access only their own personal and service records.
SEC-04	Input shall be validated before storage.
SEC-05	Location shall stop after attendance submission.
SEC-06	Payment credentials shall not be stored by HostelSpace.

2.7 Business / Domain Rules

ID	Rule
BR-01	One Chief Warden supervises multiple wardens.
BR-02	Study-room status does not support booking.
BR-03	Students cannot select or reserve a laundry day.
BR-04	Laundry collection requires an earlier received record.
BR-05	Only one accepted attendance is allowed per student per session.
BR-06	A failed payment does not mark the fee as paid.
BR-07	Deviation total is the sum of active entries.

2.8 Database / Field Requirements

Record	Main Fields
Users / Profile	user_id, login_id, password_hash, role, active, name, SRN, room, phone
Notice / Rule	record_id, title/text, active dates, rule_id, weightage
Study Room	status, timings, message, updated_by, updated_at
Laundry	student_id, allocated_day, item details/count, status, handover/collection time
Leave	leave_id, student_id, dates, reason, assigned warden, status, remarks
Attendance	attendance_id, session, student_id, submitted_at, location result, status
Complaint	complaint_id, student, room, category, description, assigned staff, status
Mess / Coupon	student_id, mess name, coupon code, validity, active status
Fee Payment	payment_id, student_id, amount, status, reference, date
Deviation	deviation_id, student_id, rule_id, weightage, remarks, recorded_by

2.8.1 Glossary

Term	Meaning
Chief Warden	Main warden who supervises other wardens.
Deviation weightage	Value attached to a recorded rule deviation.
Laundry tracker	Assigned day and clothes handover/collection record.
Location check	One-time attendance location check during submission.
Payment reference	Unique reference for a payment result.
RBAC	Permissions based on the signed-in user's role.

2.9 Functional Ownership

Member	Complete Functionality	Requirement IDs
Anvi Srinivas	Student profile; notices; hostel information and contacts; hostel rules	FR-01, FR-02, FR-11
Arin Singh	Leave; location-verified attendance; deviation-weightage records	FR-05, FR-06, FR-10
Aryan Singh	Study-room Open/Closed status; allocated laundry tracker; handover and collection	FR-03, FR-04
Atiksh Gour	Maintenance complaints; mess allocation and coupons; hostel fee payment	FR-07, FR-08, FR-09

2.9.1 Requirement Traceability Matrix

Each requirement is linked to a use case, owner, measurable acceptance result, verification method and planned references for later design, code and test documents.

ID	UC	Owner	Acceptance Result	Verification	Later Links
FR-01	UC-01	Anvi	Valid active user reaches the correct dashboard; unauthorized pages are blocked; only permitted profile fields are changed.	Login, role and profile tests	D-01 / C-01 / TC-01
FR-02	UC-02	Anvi	Authorized staff can maintain content; students see only active notices, information and rules.	Role and content tests	D-02 / C-02 / TC-02
FR-03	UC-02	Aryan	Latest Open/Closed status and optional message are shown; no booking action is available.	Status and interface tests	D-03 / C-03 / TC-03
FR-04	UC-03	Aryan	Staff assigns the day; student cannot change it; collection is blocked until a received record exists.	Workflow and validation tests	D-04 / C-04 / TC-04
FR-05	UC-04	Arin	Valid request gets an ID and Pending status; invalid dates are rejected; warden decision and escalation are stored.	Leave workflow tests	D-05 / C-05 / TC-05
FR-06	UC-05	Arin	Attendance is accepted only in the allowed time and location; only one accepted record is stored per session.	Boundary, location and duplicate tests	D-06 / C-06 / TC-06
FR-07	UC-06	Atiksh	Valid complaint gets an ID; staff can update status; student can view only personal complaints.	Complaint workflow and access tests	D-07 / C-07 / TC-07
FR-08	UC-07	Atiksh	Student sees the assigned mess and active coupons; only authorized Mess Staff can update them.	Role and record tests	D-08 / C-08 / TC-08
FR-09	UC-08	Atiksh	Success stores one unique reference and marks Paid; failure remains Unpaid; payment credentials are not stored.	Payment result, duplicate and data tests	D-09 / C-09 / TC-09
FR-10	UC-09	Arin	Entry stores rule, weightage and warden; total equals active entries; student sees only personal records.	Calculation and authorization tests	D-10 / C-10 / TC-10
FR-11	UC-10	Anvi	Only the Hostel Director can manage authorized users, settings and required summary reports.	Director-role and report tests	D-11 / C-11 / TC-11

2.1.2 References

- PES University, UE24CS341A: Software Engineering — Level 3 Student Submission Guide.
- HostelSpace agreed first-version project scope and requirements.
- Technology documentation for Next.js, JavaScript, SQL, Git/GitHub, and the selected testing/deployment tools.

2.3.2 Product Perspective

HostelSpace is a browser-based hostel management system that acts as a centralized application for student services and authorized hostel operations. It connects the browser interface with the Next.js application layer, SQL data storage, browser location services for attendance, and a payment service for fee-payment status. The system is intended to replace fragmented/manual service tracking for the defined first-version scope.

2.5 System Features — Stimulus / Response Summary

Feature	Stimulus	System Response
Authentication & profile	User submits credentials or profile changes	System validates the session/role and applies only permitted profile changes; correct dashboard is shown.
Notices, information & rules	Authorized staff creates/updates content or student requests content	System stores authorized changes and displays active content to students.
Study-room status	Authorized staff changes Open/Closed status	Latest status, timing and optional message are displayed; no booking is offered.
Laundry tracker	Staff assigns a day or records received/collected items	System stores the workflow state and shows the assigned day/status to the student.
Leave & attendance	Student submits leave or attendance	System validates leave data or checks attendance time/location and records the resulting status.
Complaints	Student submits a complaint or staff updates it	System creates/tracks the complaint and restricts student viewing to personal records.
Mess / coupons	Mess Staff updates allocation/coupons	System stores authorized changes and shows the current records to the student.
Fee payment	Student starts a fee payment and the payment service returns a result	System stores success/failure/pending status and reference without storing payment credentials.
Deviation records	Warden records a rule deviation	System stores the entry and calculates the total from active deviation entries.
Users / reports	Hostel Director manages users/settings or requests a report	System performs authorized management actions and displays the required summary report.

2.5.1 Detailed Functional Requirements

The detailed functional requirements FR-01 to FR-11 already defined in this SRS remain the authoritative requirements and are retained unchanged. Each is linked to its corresponding use case and RTM entry.

2.6.2 Use Case Specification

Use Case ID	Use Case	Primary Actor(s)	Precondition	Outcome
UC-01	Register / Login	Student	Application available	Authenticated session and permitted dashboard
UC-02	Notices / Rules	Student; authorized staff	Content exists or authorized user is signed in for changes	Active content displayed or updated
UC-03	Study-room Status	Student; authorized staff	Application available	Current status displayed or updated
UC-04	Laundry Tracker	Student; Laundry Staff	Student record exists	Assigned day/status and handover/collection recorded
UC-05	Leave Management	Student; Warden; Chief Warden	Student authenticated for submission	Leave request decision/escalation recorded
UC-06	Attendance	Student; Warden	Attendance rules configured	Attendance accepted or rejected with result recorded
UC-07	Complaints	Student; Maintenance Staff	Student authenticated	Complaint created and status tracked
UC-08	Mess / Coupons	Student; Mess Staff	Student record exists	Allocation/coupon records displayed or updated
UC-09	Fee Payment	Student	Fee record exists	Payment result/reference recorded; fee state updated appropriately
UC-10	Users / Reports	Hostel Director	Director authenticated	Authorized users/settings managed or report displayed

2.7 Other Requirements

The following requirements complement the functional, non-functional, security, and domain requirements.

Area	Requirement
Database	The SQL database shall maintain the required user, profile, service, payment, and deviation records using the field definitions specified in the field-requirements section.
Data retention	Records required for hostel operations, audit, and evaluation shall be retained according to the project's agreed institutional requirements; retention duration is outside the first-version scope unless separately specified.
Deployment	The application shall be deployable as a browser-accessible Next.js application with an SQL database and HTTPS for private communication in deployment.
Localization	The first release shall use English for user-facing content. Additional language localization is outside the current first-version scope.
Reuse	The project shall reuse suitable framework libraries and development infrastructure rather than reimplementing standard framework, testing, version-control, CI/CD, and containerization capabilities.
Compatibility	The application shall support current Chrome, Edge, and Firefox as stated in NFR-04 and shall remain usable across phone, tablet, laptop, and desktop form factors.

2.10 Validation Specification — Initial Planning

Validation is planning-focused at the requirements stage. Functional, security, and non-functional requirements will be validated using the methods below; this is not the final test report.

Category	Planned Validation	Example / Evidence
Functional	Black-box positive, negative, boundary, and role-based tests for each workflow.	Valid/invalid leave, attendance, complaint, payment workflows; RTM-linked test evidence.
Security	Authentication/authorization tests plus planned security/penetration testing.	Student attempts a warden/admin endpoint; input-validation and data-isolation checks.
Performance	Load and response-time testing under the agreed project test load.	Measure normal page response time and record environment, observed time, and pass/fail result.
Usability	Task-based walkthrough of dashboards, forms, labels, messages, and responsive use.	New student submits a complaint or leave request.
Reliability	Failure, retry, interrupted-response, and duplicate-prevention scenarios.	Payment timeout/retry and repeated attendance submission.
Unit	Automated unit tests with coverage for validation, calculation, permission, and state rules.	Deviation total, duplicate prevention, laundry collection, and payment-state logic.

2.10.1 Validation Records

For each test, the team will record the requirement ID, test case ID, preconditions, input, expected result, actual result, status, and evidence reference. Failed tests will be linked to a defect, corrected, and rerun before the final demonstration.

2.11 Project Plan

Agile lifecycle justification. Agile is suitable because HostelSpace contains independent but connected modules that can be delivered in increments. Six sprints support early review of authentication and student access, followed by service workflows, integration, security checks, defect correction, and final evidence. Faculty feedback can be incorporated between sprints without changing the agreed first-version scope or functional ownership.

Reuse/build classification. The team will build the HostelSpace application modules and SQL data model. Reusable infrastructure/components include Next.js/React capabilities, JavaScript libraries, authentication facilities, SQL database facilities, testing frameworks, Docker, GitHub Actions, Jira, and design tools.

2.11.1 Work Breakdown Structure, Ownership, and Effort

WBS	Work Package / Tasks	Owner(s)	Effort (person-days)	Output
1.0	Requirements review, backlog, repository, base project, and role model	All; Atiksh coordinates	10	Approved backlog, repository, running base
2.0	Authentication, student profile, authorized users, notices, information, contacts, and rules	Anvi	24	FR-01, FR-02, FR-11 implementation and tests
3.0	Study-room status, laundry-day allocation, handover, and collection	Aryan	18	FR-03 and FR-04 implementation and tests
4.0	Leave, warden decisions, attendance location check, and deviation records	Arin	22	FR-05, FR-06, FR-10 implementation and tests
5.0	Complaints, mess allocation/coupons, fee payment, and payment status	Atiksh	22	FR-07, FR-08, FR-09 implementation and tests
6.0	Integration, access checks, automated/manual validation, defects, documentation, and demonstration	All	20	Stable build, test evidence, final report, team demo

Total estimated effort: 116 person-days ≈ 5.8 person-months, assuming 20 working days per person-month.

2.11 Project Plan (continued)

2.11.2 Sprint Outputs and Acceptance Evidence

Sprint	Planned Output	Acceptance Evidence
1	Project setup, Jira backlog, GitHub repository, authentication, user roles, and student profile	User stories, repository activity, working sign-in/profile
2	Notices, information/rules, study-room status, and laundry-day allocation display	Working pages, sample records, tests, pull requests
3	Laundry handover/collection, leave application, and warden approval	Feature demonstration and functional test results
4	Location-verified attendance, deviation records, and complaint status workflow	Integration results and role-access checks
5	Mess allocation/coupons, fee payment/status, and required reports	Completed workflows, validation evidence, test summary
6	Final integration, defect correction, security checks, documentation, and demonstration	Stable build, final report evidence, team demo

2.11.3 Gantt Chart / Timeline

Six sequential Agile sprints are planned. Activities may overlap where practical; the chart provides the required high-level schedule.

Work Package	Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5	Sprint 6
Planning, setup, authentication	WORK					
Profile, notices, information, rules	WORK	WORK				
Study room and laundry		WORK	WORK			
Leave, attendance, deviations			WORK	WORK		
Complaints, mess, fees, reports				WORK	WORK	
Integration, validation, documentation				WORK	WORK	WORK
Defect correction and final demo						WORK

2.12 Tool Stack

Activity	Selected Tool	Evidence to Submit
Version control	Git + GitHub	Repository URL and commit history
Agile planning	Jira	Backlog, user stories, sprint board
CI/CD	GitHub Actions	Workflow YAML and successful pipeline run logs
Containerization	Docker	Dockerfile and build/run evidence
Unit/component testing	Jest + React Testing Library	Test results and coverage
End-to-end testing	Playwright or Cypress	E2E test cases and execution evidence
Static analysis	ESLint + SonarQube/SonarCloud	Lint and analysis reports
API testing	Postman or Thunder Client	Collection and test results
Design / UML	Draw.io or PlantUML	Use-case and ER/UML diagrams

The selected tools are tailored to the actual HostelSpace technology stack (Next.js + JavaScript + SQL) while satisfying the tool-infrastructure expectations of the submission guide.

Appendix A — Glossary

The glossary previously presented in the SRS is retained as Appendix A. It defines Chief Warden, deviation weightage, laundry tracker, location check, payment reference, and RBAC.

Appendix B — Field Layouts / Report Requirements

The database/field requirements previously presented in the SRS are retained as Appendix B. They specify the principal fields for users/profile, notices/rules, study room, laundry, leave, attendance, complaints, mess/coupons, fee payment, and deviations.

Appendix C — Requirement Traceability Matrix

The RTM previously presented in the SRS is retained as Appendix C. It links FR-01 through FR-11 to use cases, owners, acceptance results, verification methods, and planned design/code/test references.

Submission structure note

The appendices above are labels for the existing approved content; no requirement IDs or substantive requirement statements have been changed. The original use-case diagram and ER model remain on the analysis-model page.

PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

HostelSpace

Hostel Management System

DELIVERABLE 3

SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

Item	Details
Project Title	HostelSpace
Assigned Project	Hostel Management System
Group Number	Team 11
Course	UE24CS341A : Software Engineering
Methodology	Agile
Version	1.0
Date	29 September 2026
Status	Prepared for Review

Team Members

USN	SRN / PRN	Name
PES1UG24CS076	PES1202402178	Anvi Srinivas
PES1UG24CS079	PES1202402179	Arin Singh
PES1UG24CS086	PES1202400020	Aryan Singh
PES1UG24CS096	PES1202403356	Atiksh Gour

Revision History

Version	Date	Author	Change Summary
1.0	29-09-2026	Team 11	Initial SAD derived from approved SRS and development plan; adopts Next.js + Django REST architecture.
1.1 (planned)	After review	Team 11	Record faculty/team decisions for open parameters and implementation evidence.

Approvals

Role	Name	Signature / Date
Architecture Lead	Anvi Srinivas	
Design / Schema Lead	Arin Singh	
Integration Lead	Aryan Singh	
CI/CD Lead	Atiksh Gour	
Faculty Evaluator		

1. Introduction

1.1 Purpose

This document specifies the architecture and detailed design of HostelSpace. It translates FR-01–FR-11, NFR-01–NFR-05, SEC-01–SEC-06 and BR-01–BR-07 into implementable components, data models, interfaces, security controls, workflows and deployment decisions.

1.2 Scope

The design covers the browser portal, Next.js frontend, Django REST Framework API, PostgreSQL data store, one-time browser location check, test-mode payment integration, Docker development environment, GitHub workflow and CI/CD delivery path. It excludes study-room booking, laundry-day selection by students, continuous location tracking, full accounting, biometric devices, inventory management and a native mobile app.

1.3 Audience

Team 11 developers and testers, the faculty evaluator, reviewers, security reviewers and future maintainers.

1.4 Definitions

Term	Meaning
DRF	Django REST Framework
RBAC	Role-based access control
CSRF	Cross-Site Request Forgery protection
ORM	Django Object–Relational Mapper
RTM	Requirements Traceability Matrix
SHA	Git commit identifier used to identify a release
ADR	Architecture Decision Record

2. Document Overview

2.1 How to use this document

Section 3 defines system-wide architecture, quality attributes and security boundaries. Section 4 gives detailed module, data, API and workflow design. The appendices capture traceability, open decisions and implementation ownership. Diagrams are normative at the architectural level; exact field names may be refined through reviewed Django migrations without changing SRS behavior.

2.2 Related Documents

- HostelSpace Deliverable 2 SRS, version 1.0, 2 September 2026.
- HostelSpace Development Plan, planning version 1.0, 5 September 2026.
- HostelSpace Software Test Plan, version 1.0, 29 September 2026.
- Live Jira/GitHub issues, pull requests, CI runs and RTM evidence created during implementation.

2.3 Baseline clarifications

- The authoritative runtime path is Browser → Next.js → Django REST API → PostgreSQL; direct Next.js-to-SQL access is not permitted.
- FR-03 retains its requirement ID. The duplicate UC-02 label should be corrected to UC-11 for study-room status in the next controlled SRS revision.
- This SAD is a design specification, not evidence that code, tests or deployment have already been completed.

3. Architecture

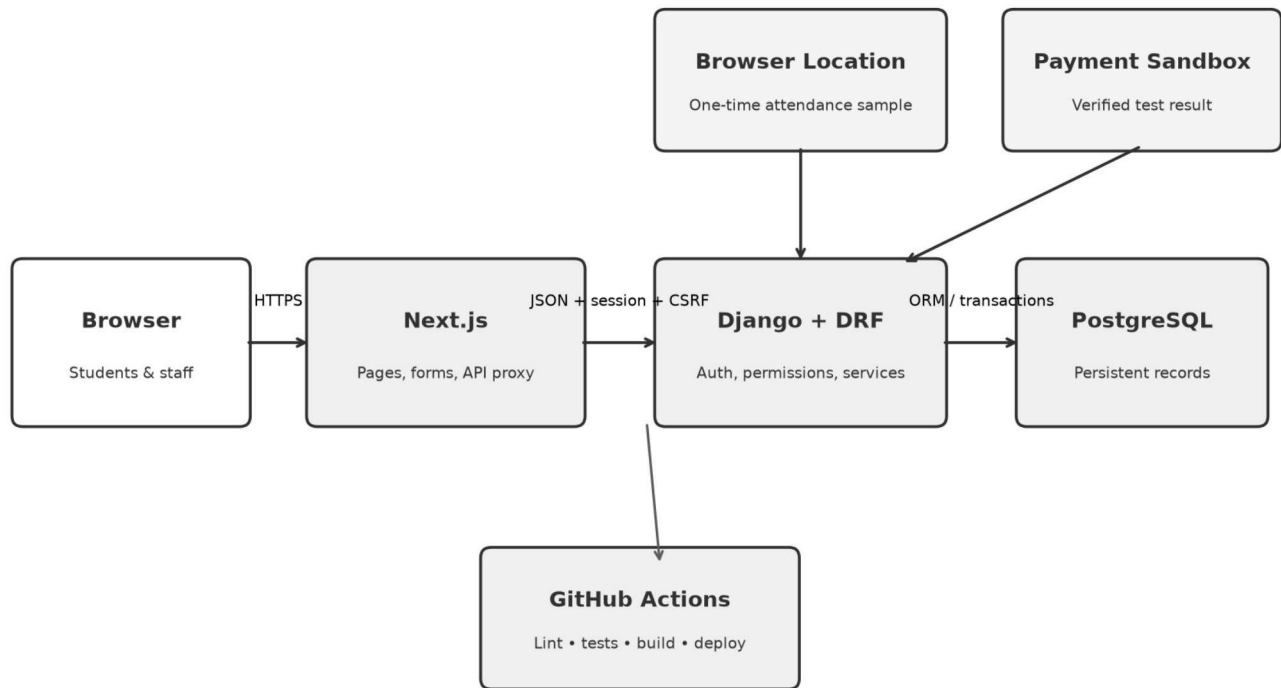
3.1 Goals & Constraints

Goal / Constraint	Design response
Security and privacy	Django sessions, CSRF, server-side RBAC and object ownership; no payment credentials; one-time location handling.
Reliability and integrity	Database constraints, transactions, idempotent transitions and verified payment results.
Testability	Modular Django apps, shared API client, unit/API/component/E2E tests and PostgreSQL integration tests.
Performance	Simple layered request path, indexed ownership/status fields and agreed warm-load test with $p95 \leq 3$ seconds.
Maintainability	Single monorepo, one frontend, one backend, one primary database, documented module ownership.
Course constraints	Agile/Jira, Git/GitHub, Docker evidence, CI/CD, traceability, documentation and role-based demonstration.
Scope constraint	No microservices, Redis, Celery, Kubernetes, booking, meal ordering or full accounting in V1.

3.2 Stakeholders & Concerns

Stakeholder	Primary concerns
Students	Simple access, privacy, clear statuses and useful error messages.
Wardens / Chief Warden	Correct assignment, escalation and audit history.
Director	Authorized-user control and accurate summary counts.
Service staff	Only assigned module actions; concise operational screens.
Team / maintainers	Modularity, reproducible setup, reviewable migrations, diagnostics and rollback.
Faculty evaluator	Traceability, working evidence, security, tests and understandable individual ownership.

3.3 Component (UML-style) Diagram



3.4 Component Descriptions

Component	Responsibilities	Key interfaces
Browser	Renders UI; requests one location sample only after user action; stores session only as secure cookie.	HTTPS pages; browser geolocation; provider-hosted payment UI.
Next.js frontend	App Router pages, forms, role-aware navigation, shared components and same-origin /api proxy.	JSON to DRF; no direct database access.
Django/DRF API	Authentication, CSRF, RBAC, ownership, validation, service workflows, serializers and health/version endpoints.	REST/JSON; ORM; payment verification.
Django services / ORM	Transaction boundaries, state transitions, calculations and portable migrations.	PostgreSQL; SQLite only as optional quick local mode.
PostgreSQL	Authoritative integration/staging/production store; constraints and indexes.	Django ORM only.
GitHub Actions	Independent delivery path for lint, security, tests, builds, staging and production gates.	GitHub events; Vercel/Render provider APIs.

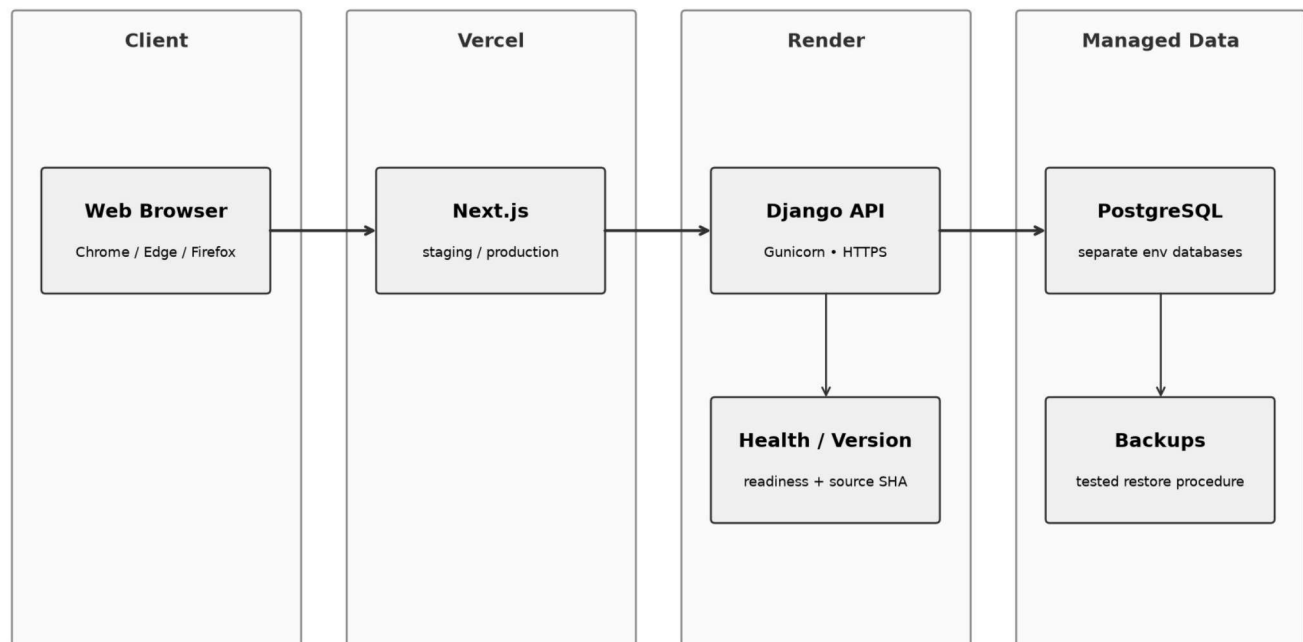
3.5 Chosen Architecture Pattern and Rationale

A modular layered monolith is selected. The presentation layer (Next.js) is separated from the API/domain layer (Django/DRF) and persistence layer (Django ORM/PostgreSQL). Django apps group related capabilities but deploy as one API. This preserves separation of concerns and parallel ownership without the operational cost and distributed consistency problems of microservices. A second authentication system, direct frontend SQL access and client-authoritative authorization are explicitly rejected.

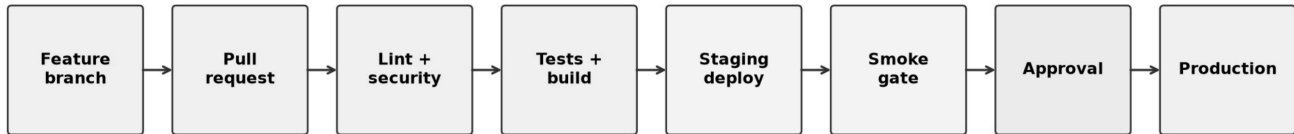
3.6 Technology Stack & Data Stores

Layer / Tool	Selection and rationale
Frontend	Next.js App Router with JavaScript and Tailwind CSS; shared forms, cards and tables.
Backend	Python 3.12 baseline, supported Django 5.2 LTS patch and compatible DRF release.
Authentication	Django sessions and secure password hashing; same-origin proxy; CSRF on every state-changing request.
Data	PostgreSQL 16 for CI/staging/production; optional SQLite only for quick local development.
Testing	pytest/pytest-django/coverage; Jest + React Testing Library; small Playwright critical-flow suite.
Quality / security	Ruff, ESLint, Bandit, pip-audit and npm audit; findings require human triage.
Containers	Frontend and backend Dockerfiles plus Compose with PostgreSQL and named volume.
Delivery	GitHub Actions; Vercel for frontend and Render for API/database, subject to current plan availability.

3.7 Deployment Architecture



3.8 CI/CD Architecture and Release Gates



Any failed gate stops promotion; production receives the reviewed source SHA.

Pull requests receive no deployment secrets. Required checks are lint, security scan, PostgreSQL backend tests, frontend tests and builds. Only a successful trusted main push may deploy staging. Health, version-SHA and synthetic smoke checks must pass before production approval. Production deploys the same reviewed source SHA. A failed gate stops promotion; rollback uses a recorded known-good frontend/API pair and never assumes application rollback reverses database migrations.

3.9 Risks & Mitigations

Risk	Impact	Mitigation
Session/cookie proxy misconfiguration	Login or CSRF failure in hosted environment	Test cookie forwarding, trusted origins and logout during the first vertical slice.
SQLite hides PostgreSQL behavior	Duplicate or transaction defects	Run migrations and uniqueness/retry tests on PostgreSQL in CI.
Browser location is spoofable or unavailable	Attendance is not tamper-proof	Treat it as a limited signal; reject unavailable/low-accuracy cases per agreed policy; retain no raw history.
Payment callback trusted incorrectly	False Paid state or duplicates	Server verification, amount/currency matching, transaction and unique provider reference.
Hosting quotas / unavailable approval feature	Deployment plan blocked	Verify plans early; use reviewed manual production promotion if required and document the limitation.
Scope growth	Incomplete core functionality	Keep V1 exclusions fixed; record change control through Jira and faculty review.

3.10 Traceability to Requirements

Requirements	Architectural realization
FR-01, SEC-01–03	Django session auth, CSRF, accounts app, role/ownership permission classes.
FR-02, FR-03	Information and facilities modules with protected staff writes and student reads.
FR-04–FR-10	Feature-owned API/service modules with validated state transitions and object scoping.
FR-11	Director-only account management and read-only aggregate summaries.
NFR-01–05	Performance test, accessible forms, uniqueness, browser matrix and automated CI tests.
SEC-04–06	Serializer validation, no retained location history, no payment credentials.

3.11 Security Architecture

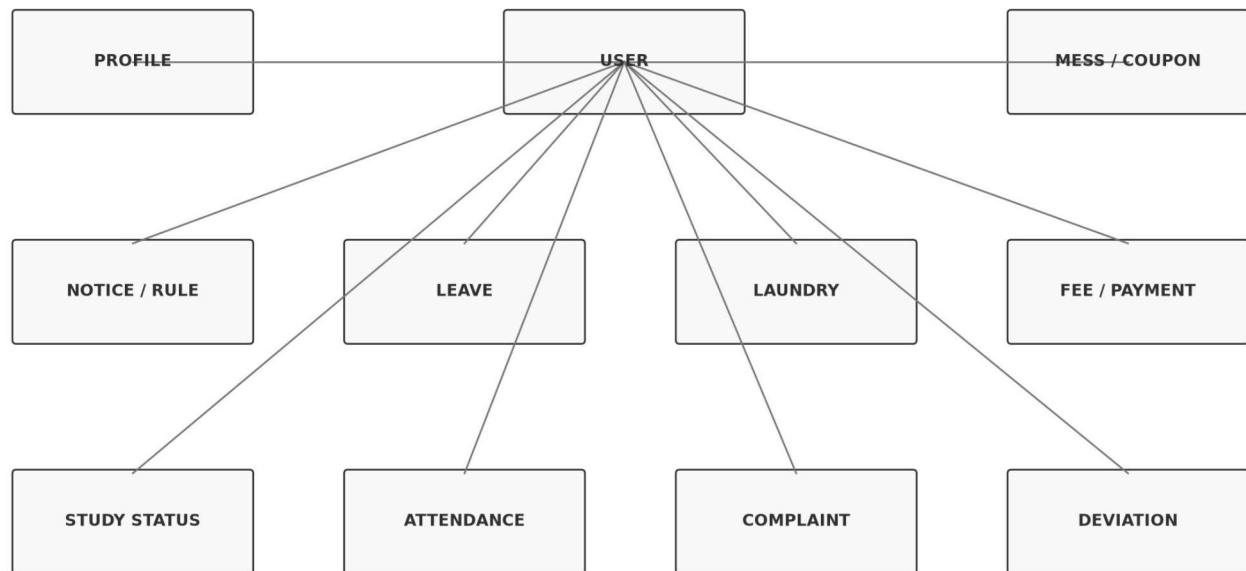
Threat (STRIDE)	Control
Spoofing	Secure password hashing, active-account checks, server sessions, HTTPS and logout invalidation.
Tampering	CSRF, server validation, ORM parameterization, signed provider verification and immutable audit fields.
Repudiation	Actor, timestamp and remarks on decisions/transitions; release SHA and PR history.
Information disclosure	Object-level scoping, no secrets in NEXT_PUBLIC/localStorage/logs, synthetic evidence only.
Denial of service	Request size limits, bounded queries, provider timeouts and health/readiness checks.
Elevation of privilege	Server-authoritative RBAC; Director-only role management; no frontend-only authorization.

4. Design

4.1 Design Overview

Each functional owner delivers a vertical slice: Next.js page/components, DRF serializer/view/permissions/service, Django model/migration and automated tests. Cross-module behavior uses documented service or query contracts rather than duplicating another module's models. Status transitions are explicit service operations, not unrestricted model updates.

4.2 Logical Data Model



Key implementation constraints: one profile per student • one active Chief Warden • one accepted attendance per student/session
unique successful payment reference • immutable historical laundry/deviation details • ownership enforced in API queries

4.2.1 Principal entities and constraints

Entity group	Principal fields	Constraints / behavior
User / StudentProfile	login, role, active; SRN, room, phone	Unique login and SRN; one profile per student; phone editable by owning student; role staff-managed.
Notice / Contact / Rule	title/text, active window, rule ID, weightage, active	Active-range validation; referenced rules deactivated rather than deleted.
StudyRoomStatus	status, timings, message, updated_by/at	Only Open or Closed; only permitted staff write; no booking entity.
LaundryAllocation / LaundryRecord	weekday; item detail/count; status; handover/collection actors/times	One current allocation; positive count; Received before Collected; repeated collect is idempotent.
LeaveRequest / LeaveTransition	dates, reason, assigned warden, status, remarks, actor/time	End ≥ start; only assigned warden decides Pending; Chief decides Escalated; final is immutable.
AttendanceSession / Attendance	window, centre, radius; student, submitted_at, result	Positive radius; valid window; unique accepted student/session; raw coordinates discarded.
Complaint / ComplaintTransition	room, category, description, assignee, status, actor/time/remarks	Ownership scoped; Open → In Progress → Resolved; Director assigns active maintenance staff.

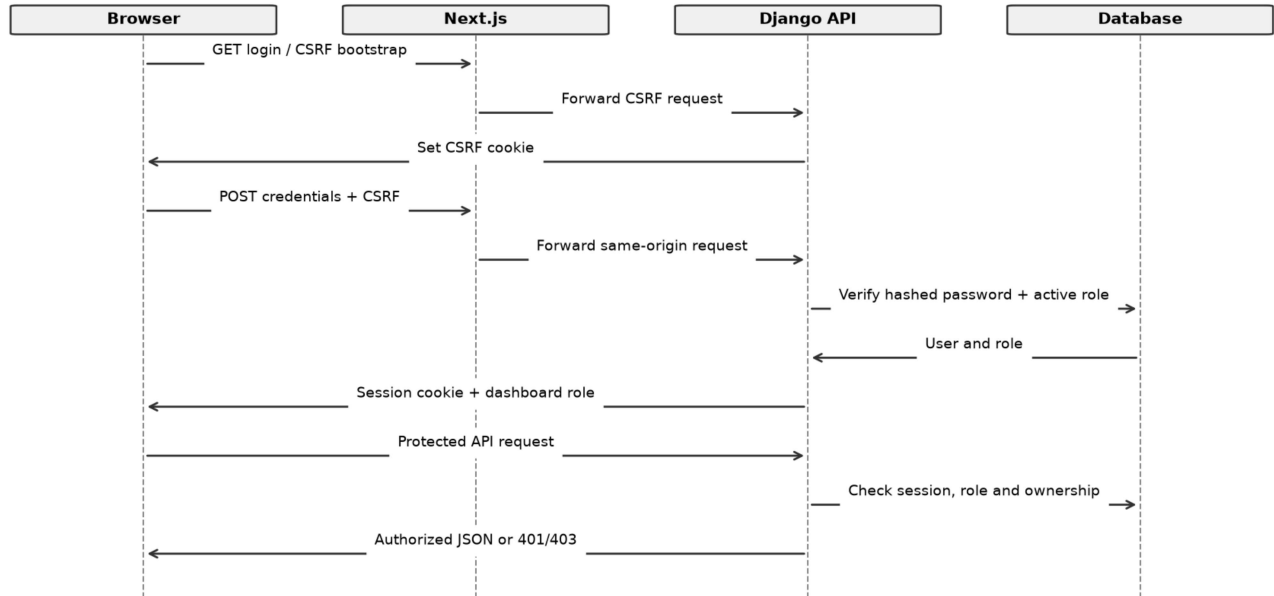
Entity group	Principal fields	Constraints / behavior
MessAllocation / Coupon	mess, code, validity, active	Valid date range; student sees current active personal records.
FeeDue / PaymentAttempt	amount/currency/status; provider reference/result	Decimal values; verified result only; unique successful reference; atomic Paid update.
DeviationRecord	student, rule, copied weightage, remarks, recorded_by, active	Historical weightage copied; total sums active entries only; personal read scope.

4.2.2 Role-permission matrix

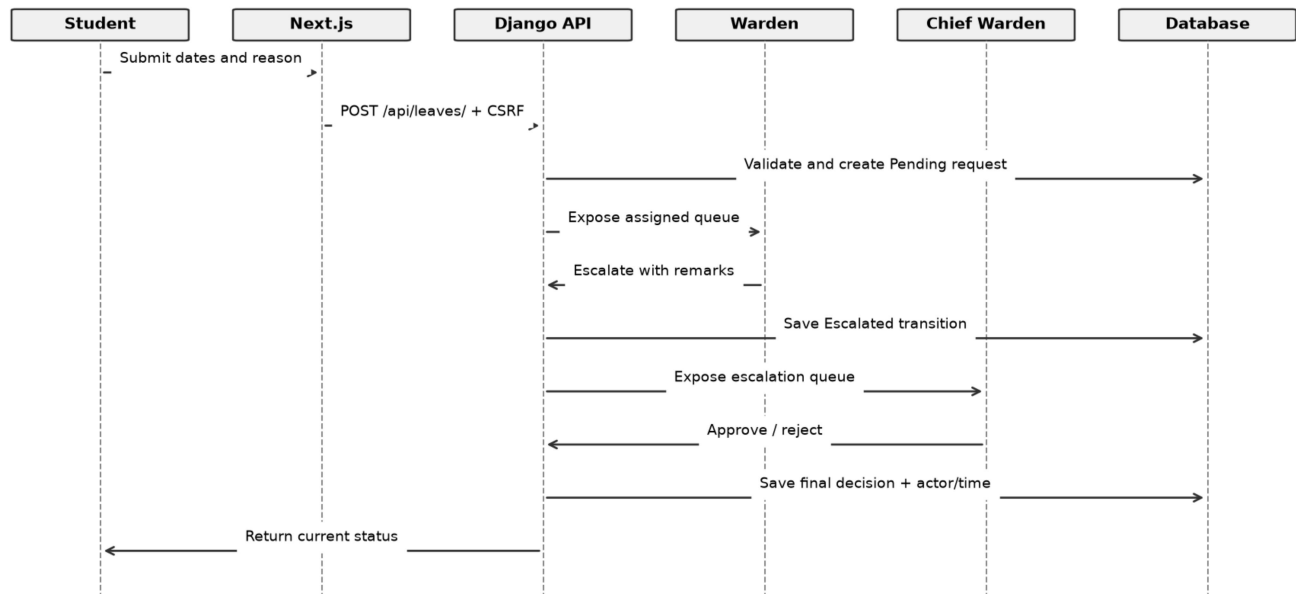
Capability	Student	Warden / Chief	Director	Service staff
Own profile	Read / phone update	No	Manage permitted accounts	No
Notices / rules	Read active	Read	Create/update/deactivate	Authorized content staff only
Study status	Read	Optional if explicitly permitted	Update	Explicitly permitted only
Leave	Submit/read own	Assigned decision / Chief escalation	Summary	No
Attendance	Submit/read own	Configure/review	Summary	No
Laundry	Read own	No	Oversight	Laundry Staff write
Complaints	Submit/read own	No	Assign/override	Assigned Maintenance Staff update
Mess / coupons	Read own	No	Oversight	Mess Staff write
Fees	View/initiate own	No	Summary	Authorized fee administration only
Deviations	Read own	Create/read authorized	Oversight	No
User roles / summaries	No	No	Director only	No

4.3 UML Sequence Diagrams

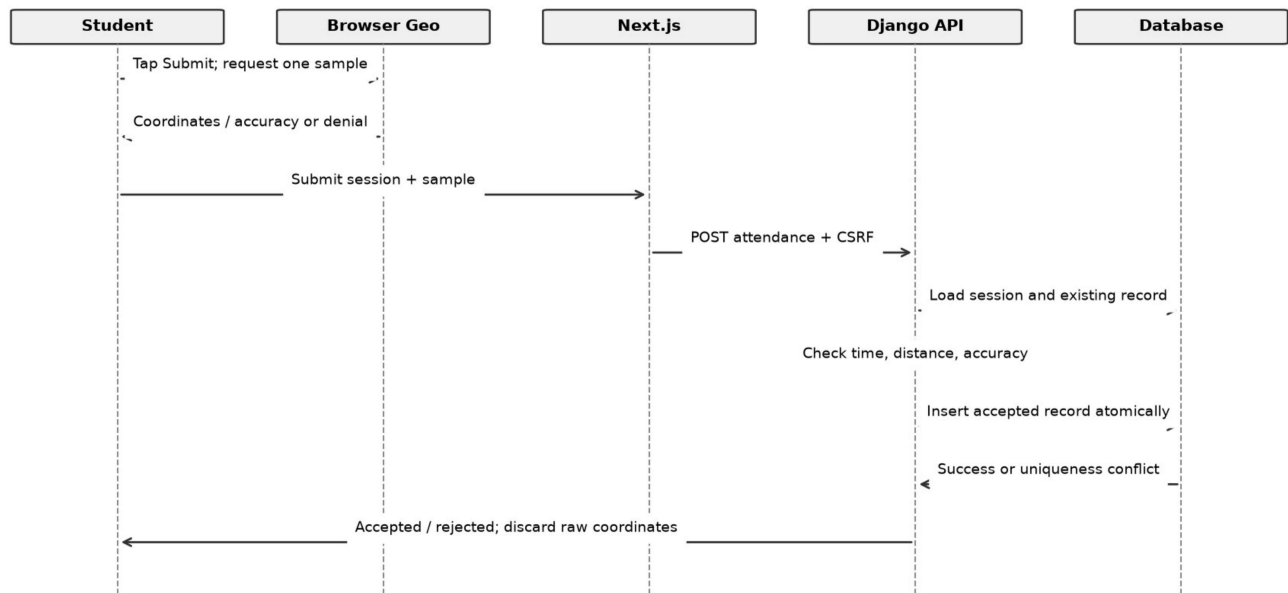
Sequence: Secure sign-in and protected request



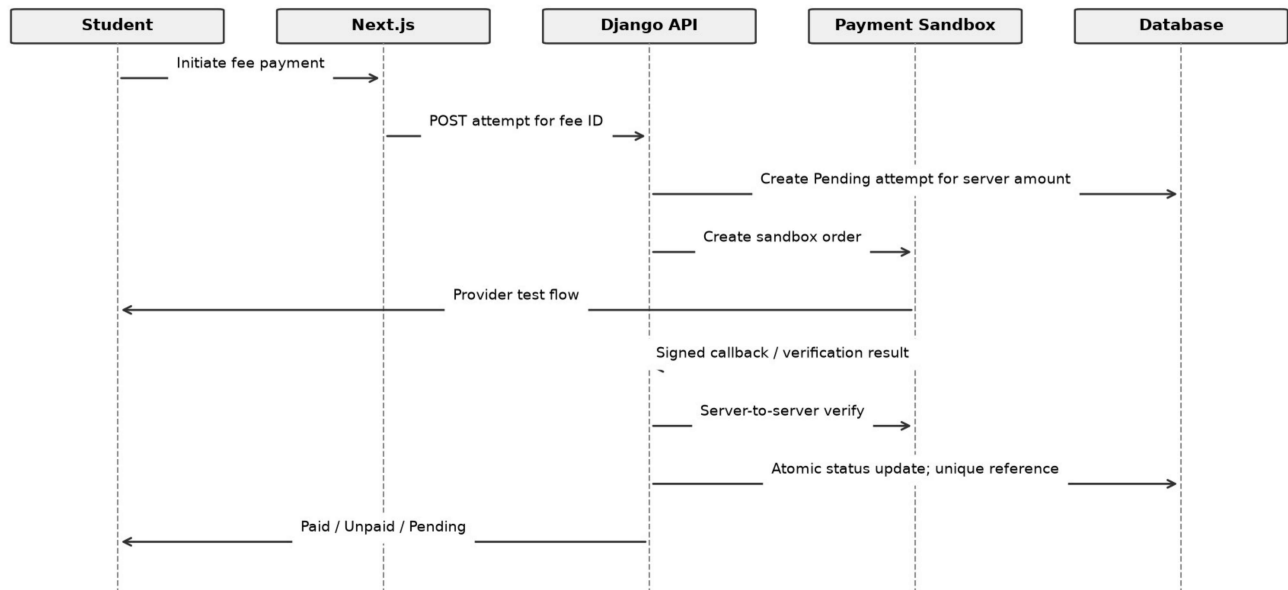
Sequence: Leave submission, escalation and decision



Sequence: One-time location attendance



Sequence: Test payment verification



4.4 API Design

All endpoints are exposed through the same-origin /api proxy. State-changing calls require an authenticated session and CSRF token. Responses use JSON and standardized error bodies: {"code": "...", "message": "...", "fields": {...}}. IDs below are opaque resource identifiers.

Requirement / Endpoint	Method	Authorized role	Input / result and key errors
FR-01 /api/auth/login; /logout; /me	POST / GET	All configured users	Credentials → session and role; 400 validation, 401 invalid/inactive, 403 CSRF.

Requirement / Endpoint	Method	Authorized role	Input / result and key errors
FR-01 /api/profile/me	GET / PATCH	Student	Return own profile; PATCH phone only; 400 malformed, 403 protected field.
FR-02 /api/information/*	GET / POST / PATCH	Reads: users; writes: Director/content staff	Active notices/rules/contacts; 400 date range/title; 403 role.
FR-03 /api/study-room/status	GET / PUT	Reads: users; writes: permitted staff	Open/Closed, timings/message; 400 invalid status; 403 role.
FR-04 /api/laundry/*	GET / POST / PATCH	Student read own; Laundry Staff write	Allocation, handover, collect; 409 invalid/repeated transition.
FR-05 /api/leaves; /{id}/decision	GET / POST	Student; assigned Warden; Chief Warden	Create Pending; approve/reject/escalate; 400 dates, 403 ownership, 409 final state.
FR-06 /api/attendance/sessions; /submit	GET / POST	Warden config; Student submit	Session and one sample; 400 time/radius, 409 duplicate accepted.
FR-07 /api/complaints; /{id}/transition	GET / POST	Student; Director; assigned Maintenance Staff	Create, assign, progress; 403 ownership; 409 invalid transition.
FR-08 /api/mess/*	GET / POST / PATCH	Student read own; Mess Staff write	Allocation/coupon; 400 validity/code.
FR-09 /api/fees; /attempts; /payment/webhook	GET / POST	Student; provider verification endpoint	Server amount; Pending/Failed/Paid; 400 mismatch, 409 duplicate reference.
FR-10 /api/deviations	GET / POST	Student read own; Warden write	Rule + copied weightage; 403 ownership/role.
FR-11 /api/director/users; /summary	GET / PATCH	Director only	Account activation/roles and aggregate counts; 409 second active Chief Warden.

4.5 Error Handling, Logging & Monitoring

- Use stable machine-readable error codes with field-level validation messages. Never expose stack traces or internal IDs in production.
- Return 401 for missing/invalid authentication, 403 for role/ownership denial, 404 for inaccessible or absent resources, 409 for state/uniqueness conflicts and 429 if rate limits are introduced.
- Structured logs include request ID, route, status, duration and authenticated user ID where appropriate, but never passwords, session cookies, CSRF tokens, raw coordinates, card/UPI data or deployment secrets.
- Health endpoint reports readiness; version endpoint reports release SHA. Monitor 5xx rate, latency, failed login spikes, payment-verification failures, migration status and deployment smoke results.

4.6 UX Design

- Role-specific navigation hides unavailable actions but never replaces backend authorization.
- Forms use persistent labels, required indicators, field-level errors, keyboard access and responsive phone/desktop layouts.
- Statuses use text plus color; empty states show “No records” or “Not assigned” rather than an error.
- Attendance explains why location is requested and handles denied/unavailable location without creating accepted attendance.
- Payment UI states test mode clearly and never requests passwords, card details or UPI PINs inside HostelSpace.

4.7 Open Issues & Next Steps

Decision	Proposed baseline	Owner / required action
Permitted profile fields	Student may edit phone only.	Team/faculty confirm before serializer finalization.
Study-room updater	Director plus explicitly permitted staff.	Record authorized role(s) in ADR and tests.
Attendance policy	Inclusive time window, positive radius; define accuracy threshold and centre/radius.	Arin to obtain team/faculty approval.
Payment provider	Use provider test mode with server verification; simulation only with explicit faculty approval.	Atiksh to obtain sandbox access.
NFR-01 load	20 signed-in simulated users; 100 warm visits; p95 ≤ 3 seconds.	Team/faculty approve test load.
UC label correction	Use UC-11 for study-room status in next SRS revision.	Anvi to update diagram, FR mapping and RTM together.

5. Appendices

5.1 Design ownership

Area	Accountable owner	Required reviewer
Accounts, information, Director UI/API	Anvi Srinivas	Arin Singh
Schema/API contracts; leave, attendance, deviations	Arin Singh	Aryan Singh
Study room, laundry, Docker and E2E	Aryan Singh	Atiksh Gour
Complaints, mess, fees, CI/CD and recovery	Atiksh Gour	Anvi Srinivas

5.2 References & Tools

- Project SRS and Development Plan supplied by Team 11.
- OWASP Application Security Verification Standard and OWASP Top 10 as security-review guidance.
- Django, DRF, Next.js, GitHub Actions, PostgreSQL, Vercel and Render official documentation.
- Diagrams prepared as architecture-level design artifacts; implementation changes require reviewed ADRs.

PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

HostelSpace

Hostel Management System

DELIVERABLE 4

SOFTWARE TEST PLAN

Item	Details
Project Title	HostelSpace
Assigned Project	Hostel Management System
Group Number	Team 11
Course	UE24CS341A : Software Engineering
Methodology	Agile
Version	1.0
Date	29 September 2026
Status	Prepared for Review

Team Members

USN	SRN / PRN	Name
PES1UG24CS076	PES1202402178	Anvi Srinivas
PES1UG24CS079	PES1202402179	Arin Singh
PES1UG24CS086	PES1202400020	Aryan Singh
PES1UG24CS096	PES1202403356	Atiksh Gour

1. Introduction

1.1 Purpose

This Software Test Plan defines objectives, scope, strategy, environment, responsibilities, cases, security validation, traceability, entry/exit criteria and reporting for HostelSpace version 1.0. It is a planned validation specification. Test results and evidence must be added only after execution against an identified build.

1.2 Scope

Testing covers FR-01–FR-11, NFR-01–NFR-05, SEC-01–SEC-06 and BR-01–BR-07 across the Next.js frontend, Django/DRF API and PostgreSQL integration environment. It includes role/ownership, workflow, input, retry, browser, usability, performance, security, Docker and CI/CD smoke validation.

1.3 References

- HostelSpace SRS v1.0, 2 September 2026.
- HostelSpace SAD v1.0, 29 September 2026.
- HostelSpace Development Plan v1.0, 5 September 2026.
- Live RTM, Jira issues, GitHub PRs, CI runs and defect records created during execution.

1.4 Definitions

Term	Meaning
UAT	User Acceptance Testing
E2E	End-to-end browser test
RTM	Requirements Traceability Matrix
CSRF	Cross-Site Request Forgery protection
p95	95th-percentile observed response/navigation time
Severity	Impact of a defect
Priority	Urgency of fixing a defect

2. Test Items

- Accounts, session authentication, profile and role navigation
- Notices, contacts, rules and study-room status
- Laundry allocation, handover and collection
- Leave submission, assignment, escalation and final decision
- Attendance-session configuration and one-time location submission
- Complaints, assignment and status transitions
- Mess allocations and coupons
- Fee dues, payment attempts and verified sandbox results
- Deviation records and totals
- Director user management and summary dashboard
- Docker Compose environment, CI pipeline, staging and release smoke checks

3. Features to be Tested

Requirement group	Coverage objective
FR-01–FR-11	All SRS functional acceptance results, positive and negative paths, roles, ownership and state transitions.
NFR-01	Agreed warm load: proposed 20 signed-in simulated users and 100 warm visits; p95 navigation-to-usable-page ≤ 3 seconds.
NFR-02	Clear labels, required indicators, field-level correction messages and usable keyboard/phone/desktop layouts.

Requirement group	Coverage objective
NFR-03	Attendance and payment retries create no duplicate accepted/success records.
NFR-04	Current Chrome, Edge and Firefox; automated modules remain independently testable.
NFR-05	Important validation/calculation rules have automated tests and CI evidence.
SEC-01–06	Session, RBAC, ownership, password handling, input validation, location stop and payment-data exclusion.
BR-01–07	Chief Warden uniqueness, no booking/day selection, transition preconditions, duplicate rules and totals.

4. Features Not to be Tested

- Study-room booking or seat reservation; student laundry-day selection; meal ordering; inventory and full accounting.
- Continuous/background location tracking (explicitly prohibited rather than tested as a feature).
- Payment provider internals or live financial settlement; only the team's sandbox integration and server verification are tested.
- Vercel, Render, PostgreSQL, browser or GitHub platform internals beyond observable integration behavior.
- Real student data, real funds, production secrets or destructive tests against production.

5. Test Approach / Strategy

5.1 Levels

Level	Scope	Primary tools / owner
Unit	Model validation, services, calculations and pure UI component behavior.	pytest; Jest/RTL — feature owner
API / integration	DRF permissions, serializers, transitions, PostgreSQL constraints and provider adapters.	pytest-django + PostgreSQL — feature owner
System / E2E	Critical role-based workflows across browser, frontend, API and database.	Playwright — Aryan coordinates
Security validation	Session, CSRF, RBAC, ownership, data minimization and dependency findings.	pytest/manual/scanners — Arin coordinates
Performance / compatibility	Warm navigation time and Chrome/Edge/Firefox responsive checks.	Playwright/lightweight load tool/manual — Aryan + Anvi
Acceptance	Requirement demonstration and faculty/team sign-off against final candidate.	Staging + RTM — all owners

5.2 Test types

- Functional positive, negative and boundary testing
- Regression testing on every reviewed merge and release candidate
- Authorization and object-ownership testing with seeded cross-user data
- State-transition and idempotency testing
- Input validation and error-message testing
- Browser compatibility, responsive layout and usability testing
- Performance and reliability testing
- Static analysis, dependency audit and configuration review
- Backup/restore, rollback and deployment smoke testing

5.3 Entry Criteria

- SRS and SAD baselines reviewed; open assumptions identified.
- Versioned build available with release SHA; migrations apply to a clean PostgreSQL database.

- Fictional seed users exist for Student, Warden, Chief Warden, Director, Maintenance, Mess and Laundry roles.
- Test environment is isolated from production; provider sandbox and browser location mocks/controls are available.
- Test cases have expected results and requirement links; no result is pre-filled as Passed.

5.4 Exit Criteria

- 100% of planned critical/high-priority cases executed for the release candidate.
- All FRs, NFRs and SEC requirements have at least one executed case and evidence link.
- No open Critical or High defect; Medium defects require documented acceptance and workaround.
- Critical E2E, PostgreSQL uniqueness, security access and staging smoke cases pass.
- Known limitations, test environment, data set, failures and deferred cases are reported honestly.

5.5 Security Validation

Security area	Planned validation
Authentication / session	Invalid and inactive accounts denied; logout invalidates session; protected API rejects missing session; Secure/HttpOnly cookie in HTTPS deployment.
CSRF	Missing, invalid and cross-origin CSRF tokens are rejected on login and state-changing requests.
Authorization	Every role boundary and another-student object reference returns 403/404 without data leakage.
Password handling	Django hashing is used; password never returned, logged or stored in plain text.
Input / injection	Server validation, over-posting attempts, long/unexpected values and basic injection payloads do not bypass ORM/serializer rules.
Location privacy	No background watcher; denied/unavailable/low-accuracy flows handled; raw coordinates absent from stored records and logs after check.
Payment security	No card/UPI credentials collected; callback signature/result verified server-side; amount/currency/fee matched; duplicate result idempotent.
Secrets / configuration	No secrets in repository, browser bundles, screenshots or logs; DEBUG off, HTTPS and exact trusted origins in deployed environment.
Dependencies / static checks	Ruff, ESLint, Bandit, pip-audit and npm audit executed; findings triaged rather than automatically declared safe.

6. Test Environment

Area	Configuration
Local quick mode	Node 22 LTS; Python 3.12; Next.js; Django/DRF; optional SQLite for fast developer checks only.
Integration / CI	Ubuntu GitHub runner, PostgreSQL 16 service, clean migrations, pinned dependencies, synthetic seed data.
System / staging	Vercel staging frontend + Render staging API + separate staging PostgreSQL database; HTTPS; release SHA endpoints.
Browsers / viewports	Current Chrome, Edge and Firefox; representative phone (~390 px), tablet and desktop widths.
Test tools	pytest, pytest-django, pytest-cov, Jest, React Testing Library, Playwright, Ruff, ESLint, Bandit, pip-audit, npm audit.
Data	Fictional users and records only. Never copy production/private student data into test environments.

7. Test Schedule

Window	Planned activity	Exit evidence
Week 1 (7 days)	Case design, environment, auth/CSRF vertical slice, CI baseline	Reviewed cases; clean PostgreSQL run; CI link.
Week 2 (7 days)	Profile, information, study, laundry allocation, leave and complaints	Feature/API/unit evidence; role and invalid-input tests.

Window	Planned activity	Exit evidence
Week 3 (7 days)	Attendance, deviations, laundry completion, mess, fees and Director summary	All FRs represented on staging; uniqueness tests.
Week 4 (7 days)	System, security, browser, performance and payment edge cases	Executed results; defect links; scanner/coverage artifacts.
Week 5 (7 days)	Final regression, backup/restore, rollback, RTM retest, acceptance, documentation and release smoke.	Release candidate; recovery evidence; final summary, limitations and approved release SHA.

8. Test Deliverables

- Software Test Plan (this document)
- Version-controlled test cases and scripts
- Synthetic test-data seeding instructions
- CI test, coverage, lint and security artifacts
- Manual execution logs and browser compatibility checklist
- Jira defect reports with retest links
- Requirements Traceability Matrix with actual Jira/PR/test/evidence references
- Final Test Summary Report and release recommendation

9. Roles and Responsibilities

Role / Member	Responsibilities
Test coordination — Arin Singh	Maintain validation specification, security/access cases and RTM consistency.
System/E2E — Aryan Singh	Maintain Playwright, Docker clean-run, PostgreSQL retries and performance evidence.
Usability / acceptance — Anvi Srinivas	Browser/responsive checks, form clarity and final requirement demonstration.
CI/CD / recovery — Atiksh Gour	CI artifacts, payment edge cases, staging smoke, backup/restore and rollback.
Feature owners — all	Design and execute unit/API/component tests for owned modules; fix defects and preserve evidence.
Independent reviewer	Review tests and evidence; author cannot approve own PR or release change.

10. Risks and Mitigation

Risk	Mitigation
Stable build delayed	Start case design early; smoke small vertical slices; leave incomplete stories open.
Environment or provider downtime	Keep local Compose setup and mocked adapter tests; reschedule sandbox-only cases without inventing results.
Location variability	Use controlled browser mocks for boundary logic and a documented real-device exploratory check.
Insufficient realistic data	Use deterministic fictional seed sets covering each role, status and ownership boundary.
Flaky E2E tests	Use stable test IDs, explicit readiness, isolated data and bounded retries only for environment readiness.
Coverage target distorts quality	Treat 80% as proposed, not faculty-mandated; prioritize requirement, branch and risk coverage.
Deadline pressure	Reduce optional graphics/automation first, never authorization, validation or evidence integrity.

11. Assumptions & Dependencies

- Team/faculty confirms the NFR-01 load and attendance accuracy policy before final performance/privacy execution.
- Payment provider sandbox credentials and webhook/verification documentation are available; otherwise a faculty-approved simulation is clearly labelled.
- GitHub plan supports the intended branch/environment controls or the team records a reviewed manual alternative.
- All test results identify build SHA, environment, browser, dataset and execution time.

12. Suspension & Resumption Criteria

Suspend when	Resume when
Environment unavailable or wrong release SHA prevents trustworthy execution.	Environment is healthy and version endpoints match the intended build.
Migration or seed failure blocks more than 30% of planned cases.	Blocking setup defect is fixed and clean setup is verified.
Authentication/authorization critical defect risks exposing other users' data.	Fix is reviewed and focused security regression passes.
Payment test unexpectedly targets live mode or secrets appear in output.	Live path is disabled, secrets rotated/sanitized and security review approves resumption.
Build has a Critical defect that invalidates downstream results.	New build is deployed and smoke test passes.

12.1 Defect severity and release treatment

Severity	Definition	Release treatment
Critical	Data exposure, live-payment risk, unrecoverable corruption, or core system unavailable.	Stop affected testing and block release until fixed and retested.
High	Major requirement, authorization or workflow failure without safe workaround.	Block release unless formally accepted by faculty/product owner.
Medium	Partial failure with a safe workaround; limited usability or compatibility impact.	Fix if feasible; otherwise document owner, workaround and target.
Low	Cosmetic or minor documentation issue with no material workflow impact.	May defer with traceable issue and honest limitation.

13. Test Cases — Functional

Case ID	Trace	Preconditions	Steps / scenario	Expected result	Priority
TC-FR01-01	FR-01	Active Student; seeded profile	Sign in; open profile; update valid phone; sign out; retry protected page.	Correct dashboard; own profile only; phone persists; post-logout request denied.	High
TC-FR01-02	FR-01 / SEC-01	Inactive user and invalid password	Attempt login with invalid then inactive credentials.	Both denied without revealing which account property failed.	High
TC-FR02-01	FR-02	Student and authorized content staff; dated notices	Create valid/invalid notices; view as student; deactivate referenced rule.	Only active-window content shown; invalid dates rejected; student cannot write; rule retained inactive.	High
TC-FR03-01	FR-03 / BR-02	Permitted staff and Student	Save Open with message; try invalid value and student update; inspect UI.	Latest status shown; invalid/unauthorized writes rejected; no booking action/API.	Medium
TC-FR04-01	FR-04 / BR-03/04	Laundry Staff and two Students	Assign day; create handover; collect; repeat collection; attempt student change/cross-read.	Positive count required; history preserved; repeat idempotent; student read-only and own records only.	High
TC-FR05-01	FR-05	Student, assigned/unrelated Wardens, Chief Warden	Submit valid/invalid dates; decide or escalate; Chief finalizes; retry final decision.	Pending created only for valid input; role/assignment enforced; final transition immutable.	High
TC-FR06-01	FR-06 / BR-05	Open attendance session in PostgreSQL	Submit inside/outside time and radius; deny location; retry concurrently.	Exactly one accepted record for valid case; invalid cases not accepted; no duplicate.	Critical
TC-FR07-01	FR-07	Two Students, Director, assigned Maintenance Staff	Create complaint; cross-read; assign; progress to Resolved; attempt revert.	Own access only; valid actors/transitions; audit history retained; no silent revert.	High
TC-FR08-01	FR-08	Mess Staff and two Students	Create allocation/coupons including expired/invalid ranges; cross-read/update.	Current active own records only; invalid range rejected; student cannot write.	Medium
TC-FR09-01	FR-09 / BR-06	Fee due and provider sandbox	Initiate; verify success/failure/pending/mismatch; repeat success callback.	Only verified matching success marks Paid once; failure stays Unpaid; unverified Pending; no credentials stored.	Critical
TC-FR10-01	FR-10 / BR-07	Warden, two Students, active/inactive deviations	Record deviation; change source rule; deactivate entry; cross-read.	Historical weight unchanged; total sums active only; student sees own data; writes denied.	High
TC-FR11-01	FR-11 / BR-01	Director and non-Director	Manage active roles; attempt second active Chief; compare summary to seed data.	Director only; one active Chief; accurate counts; empty sets return zero.	High

13.1 Test Cases — Non-functional & Security

Case ID	Trace	Preconditions	Steps / scenario	Expected result	Priority
TC-NFR01-01	NFR-01	Approved load; warm staging; synthetic users	Run 20 signed-in users and 100 warm visits; measure navigation-to-usable-page.	p95 ≤ 3s; environment/dataset/run time/failures recorded.	High
TC-NFR02-01	NFR-02	Representative forms	Keyboard and phone/desktop review of labels, required fields and errors.	Every field labelled; errors actionable; no overlap/horizontal clipping.	Medium
TC-NFR03-01	NFR-03	PostgreSQL; retry/concurrency harness	Repeat attendance and verified payment requests concurrently.	One accepted attendance and one successful reference only.	Critical
TC-NFR04-01	NFR-04	Current Chrome, Edge, Firefox	Run login, form and status smoke flows at phone and desktop widths.	Core flows usable in all three browsers; defects logged.	High
TC-NFR05-01	NFR-05	CI on release candidate	Inspect required validation/calculation tests and CI reports.	Important rules have automated tests; all required jobs pass.	High
TC-SEC01-01	SEC-01/03	Two students and unrelated staff	Alter resource IDs and call endpoints directly for every personal module.	403/404 with no object data leakage.	Critical
TC-SEC02-01	SEC-02	Seeded account and database access in test env	Inspect password storage and responses/logs.	Strong Django hash; no plain password in DB, response or logs.	Critical
TC-SEC04-01	SEC-04	All create/update endpoints	Submit missing, malformed, over-posted and long values.	Server rejects invalid input and protected fields without partial write.	High
TC-SEC05-01	SEC-05	Browser instrumentation / test logs	Submit/deny attendance; inspect watchers, storage and logs.	No background watcher; no raw coordinate history after decision.	Critical
TC-SEC06-01	SEC-06	Payment sandbox	Inspect HostelSpace forms, database, logs and evidence.	No card details, UPI PIN or provider credentials collected/stored.	Critical
TC-CICD-01	NFR-05	PR with intentional failing test	Open PR; inspect secrets/gates; then fix and merge.	PR has no deploy secrets; failure blocks merge; trusted main alone reaches staging.	High
TC-REL-01	Recovery	Known-good staging SHA and synthetic backup	Restore backup separately; deploy bad candidate; roll back compatible app pair.	Record counts match; rollback restores known-good version; evidence identifies SHAs.	High

13.2 Defect management

Each failed case creates a Jira Bug linked to the requirement and story. Record build SHA, environment, preconditions, reproducible steps, expected/actual result, severity, evidence and owner. A defect is closed only after the fix is reviewed and the original case plus affected regression cases pass. Never mark an unexecuted or blocked case as Passed.

14. Test Case Management & Traceability

Requirement	Story	Test case(s)	Initial status	Evidence field
FR-01	US-01/02	TC-FR01-01/02	Not Run	Add actual Jira / PR / CI / screenshot link after execution
FR-02	US-03	TC-FR02-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
FR-03	US-09	TC-FR03-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
FR-04	US-10/11/12	TC-FR04-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
FR-05	US-05/06	TC-FR05-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
FR-06	US-07	TC-FR06-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
FR-07	US-13/14	TC-FR07-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution

Requirement	Story	Test case(s)	Initial status	Evidence field
FR-08	US-15	TC-FR08-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
FR-09	US-16	TC-FR09-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
FR-10	US-08	TC-FR10-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
FR-11	US-04	TC-FR11-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
NFR-01	US-23	TC-NFR01-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
NFR-02	US-21	TC-NFR02-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
NFR-03	US-23	TC-NFR03-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
NFR-04	US-21/23	TC-NFR04-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
NFR-05	US-20/22/23	TC-NFR05-01 / TC-CICD-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
SEC-01/03	US-01/02/05–16/22	TC-SEC01-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
SEC-02	US-01/19	TC-SEC02-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
SEC-04	All feature stories	TC-SEC04-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
SEC-05	US-07/22	TC-SEC05-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution
SEC-06	US-16/24	TC-SEC06-01	Not Run	Add actual Jira / PR / CI / screenshot link after execution

15. Test Metrics & Reporting

Metric	Definition / use
Execution progress	Executed planned cases ÷ total planned cases; blocked reported separately.
Pass rate	Passed executed cases ÷ executed cases; never include Not Run.
Requirement coverage	Requirements with executed evidence ÷ total requirements.
Defect profile	Open defects by severity, module, age and requirement.
Automation coverage	Critical/high cases automated where stable; manual-only rationale documented.
Code coverage	Backend/frontend statement and branch coverage as supporting signal; proposed threshold 80%, subject to team approval.
Performance	p50/p95/max time, error rate, environment, dataset and test window.
Release readiness	Required checks, critical cases, open risks, known limitations and recommended decision.

15.1 Reporting cadence

- CI status on every PR and trusted main push.
- Sprint review: executed cases, defects, RTM and evidence links.
- Release candidate: complete Test Summary Report with build SHA and recommendation.

16. Approvals

Role	Name	Signature / Date
Test Coordinator	Arin Singh	
System Test Lead	Aryan Singh	
Development / Usability Lead	Anvi Srinivas	
Release / CI Lead	Atiksh Gour	
Faculty / Product Owner		